第一部分

基础知识

第2章

线程安全性

你或许会感到奇怪，线程或者锁在并发编程中的作用，类似于铆钉和工字梁在土木工程中的作用。要建筑一座坚固的桥梁，必须正确地使用大量的铆钉和工字梁。同理，在构建稳健的并发程序时，必须正确地使用线程和锁。但这些终归只是一些机制。要编写线程安全的代码，其核心在于要对状态访问操作进行管理，特别是对共享的（Shared）和可变的（Mutable）状态的访问。

从非正式的意义上来说，对象的状态是指存储在状态变量（例如实例或静态域）中的数据。对象的状态可能包括其他依赖对象的域。例如，某个 HashMap 的状态不仅存储在 HashMap 对象本身，还存储在许多 Map.Entry 对象中。在对象的状态中包含了任何可能影响其外部可见行为的数据。

“共享”意味着变量可以由多个线程同时访问，而“可变”则意味着变量的值在其生命周期内可以发生变化。我们将像讨论代码那样来讨论线程安全性，但更侧重于如何防止在数据上发生不受控的并发访问。

一个对象是否需要是线程安全的，取决于它是否被多个线程访问。这指的是在程序中访问对象的方式，而不是对象要实现的功能。要使得对象是线程安全的，需要采用同步机制来协同对对象可变状态的访问。如果无法实现协同，那么可能会导致数据破坏以及其他不该出现的结果。

当多个线程访问某个状态变量并且其中有一个线程执行写入操作时，必须采用同步机制来协同这些线程对变量的访问。Java 中的主要同步机制是关键字 synchronized，它提供了一种独